Attendance Code: 359555

CMP2028M

Natural Language Processing

Week 2: “N-gram Language Models”

Heriberto Cuayahuitl

https://staff.Lincoln.ac.uk/hcuayahuitl

Agenda

- Week 1, 5-6 Feb: Tokenization and regular expressions
- Week 2, 12-13 Feb: **N-gram language models**
- Week 3, 19-20 Feb: Text classification
- Week 4, 27-27 Feb: Vector semantics and embeddings
- Week 5, 5-6 Mar: Large Language Models (LLMs)
- Week 6, 12-13 Mar: Question answering & information retrieval
- Week 7, 19-20 Mar: Constituent and dependency parsing
- Week 8, 26-27 Mar: Information extraction
- Week 9, 23-24 Apr: Overview of neural networks
- Week 10, 30 Apr-1May: Machine translation
- Week 11, 7-8 May: Chatbots and dialogue systems
- Week 12, 14-15 May: Speech recognition and synthesis

Easter break

Last Week

- ***Tokenization*** is a key stage for processing raw text into units called *tokens*. Types of tokens: words, morphemes, characters, and subwords.
    - Byte-Pair Encoding (BPE) algorithm

- **Regular expressions**: notations for characterising a set of strings. Useful for data pre-processing and information extraction or standardisation.
    - `re.search(pattern,string)`
    - `re.sub(pattern, repl, string)`

Today

- **N-Grams**
- How to implement them
- Practical considerations

Acknowledgement: Images from https://web.stanford.edu/~jurafsky/slp3/3.pdf

N-Grams

- N-grams are probabilistic models of words or tokens.
- Example:

$$P(\texttt{blue}|\texttt{The water of Walden Pond is so beautifully}) = \frac{C(\texttt{The water of Walden Pond is so beautifully blue})}{C(\texttt{The water of Walden Pond is so beautifully})}$$

- This can be done via the chain rule of probability:

$$P(w_{i:n}) = P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1})$$

or

$$P(w_{1:n}) = \prod_{k=1}^{n} P(w_k|w_{1:k-1})$$

Do we Need N-Grams?

- N-grams dominated the topic of Language Modelling (LMs) until deep learning took over.
- They are a simple but scalable form of LMs.
- Applications:
  - Spell checking and grammar correction
  - Language identification
  - Authorship attribution
  - Text generation
  - Speech recognition (to rank hypotheses)
  - …

The Markov Assumption

- Instead of calculating the probability of a word given its entire history, we can use the last few words, i.e.

$$P(\texttt{blue}|\texttt{The water of Walden Pond is so beautifully})$$



$$P(\texttt{blue}|\texttt{beautifully})$$

- Thus, N-grams are:
  - Unigrams: $P(w_n|w_{1:n-1}) = P(w_n)$
  - Bigrams: $P(w_n|w_{1:n-1}) = P(w_n|w_{n-1})$
  - Trigrams: $P(w_n|w_{1:n-1}) = P(w_n|w_{n-2}, w_{n-1})$
  - Fourgrams: $P(w_n|w_{1:n-1}) = P(w_n|w_{n-3}, w_{n-2}, w_{n-1})$
  - …

Today

- N-Grams
- **How to implement them**
- Practical considerations

Acknowledgement: Images from https://web.stanford.edu/~jurafsky/slp3/3.pdf

How to Estimate Probabilities?

- We can use a proceduce called 'Maximum Likelihood Estimation (MLE)', which is based on word counts.

- For bigrams: $P(w_n|w_{n-1}) = \frac{C(w_{n-1}\, w_n)}{C(w_{n-1})}$

- For any n-gram: $P(w_n|w_{n-N+1}) = \frac{C(w_{n-N+1}\, w_n)}{C(w_{n-N+1})}$

2-gram, 3-gram, etc.

MLE: Example (1/2)

- Consider the following tiny corpus, where <s> and </s> are special symbols:

<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>

- Here some **unigrams** from that corpus:



Unigram	Probability
<s>	3/20
</s>	3/20
I	3/20
am	2/20
Sam	2/20
do	1/20
not	1/20
like	1/20
green	1/20
eggs	1/20
and	1/20
ham	1/20

MLE: Example (2/2)

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

- Here some **bigrams** from that corpus:

$$P(\text{I}|\text{<s>}) = \frac{2}{3} = 0.67 \qquad P(\text{Sam}|\text{<s>}) = \frac{1}{3} = 0.33 \qquad P(\text{am}|\text{I}) = \frac{2}{3} = 0.67$$
$$P(\text{</s>}|\text{Sam}) = \frac{1}{2} = 0.5 \qquad P(\text{Sam}|\text{am}) = \frac{1}{2} = 0.5 \qquad P(\text{do}|\text{I}) = \frac{1}{3} = 0.33$$

- Here some **trigrams** from the same corpus:

$$P(\text{am}|<\text{s}>,\text{I}) = 1/14 \quad P(\text{do}|<\text{s}>,\text{I}) = 1/14$$
$$P(\text{Sam}|\text{I},\text{am}) = 1/14 \quad P(\text{like}|\text{do},\text{not}) = 1/14$$
$$P(</\text{s}>|\text{I},\text{am}) = 1/14 \quad P(\text{eggs}|\text{like},\text{green}) = 1/14$$

N-Grams as Bayesian Networks

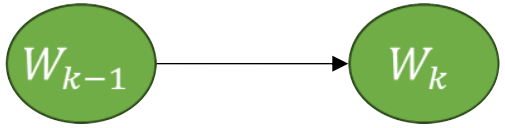
N-grams can be represented as Bayesian nets, where each of their node contains a conditional probability table:

- Unigrams:



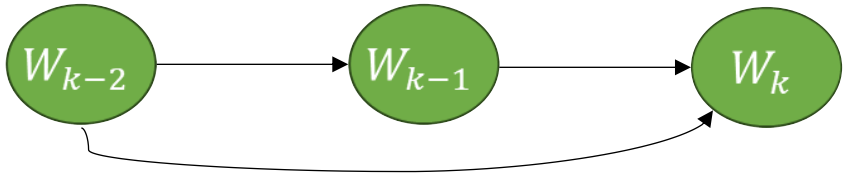
One table for $P(w_k)$, as in our n-grams example.

- Bigrams:



Two tables for $P(w_{k-1})$ and $P(w_k|w_{k-1})$

- Trigrams:



Three tables for
$P(w_{k-2})$
$P(w_k|w_{k-1})$
$P(w_k|P_{w-2}, w_{k-1})$

They can be used for probabilistic inference, i.e., to calculate probabilities $P(w_{1:n}) = \prod_{k=1}^{n} P(w_k|parents(w_k))$

Same equation as in slide 5!

Computing the Probability of Sentences

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

$P(\texttt{i}|\texttt{<s>}) = 0.25$ $\quad$ $P(\texttt{english}|\texttt{want}) = 0.0011$

$P(\texttt{food}|\texttt{english}) = 0.5$ $\quad$ $P(\texttt{</s>}|\texttt{food}) = 0.68$

$$
\begin{aligned}
&P(\texttt{<s> i want english food </s>}) \\
&\quad = P(\texttt{i}|\texttt{<s>})P(\texttt{want}|\texttt{i})P(\texttt{english}|\texttt{want}) \\
&\qquad\qquad P(\texttt{food}|\texttt{english})P(\texttt{</s>}|\texttt{food}) \\
&\quad = 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 \\
&\quad = 0.000031
\end{aligned}
$$

Evaluating N-Grams

- A better language model is better at predicting next words when they occur in a test set (less surprised).

- A perfect language model would correctly guess each next word in a corpus, with a probability of 1, and all the others with a probability of 0.

- The most common metric is **perplexity** (PP or PPL):

$$\text{perplexity}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_1 \ldots w_{i-1})}} = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

- The lower the perplexity on data the better the model.

Perplexity: Example

- Assume a tiny vocabulary $\{red, blue, green\}$ ... and two language models:
  - Model A: $P(red) = P(green) = P(blue) = 1/3$
  - Model B: $P(red) = 0.8, P(green) = 0.1, P(blue) = 0.1$

- The Perplexities are:
  - Test set: “red red red red blue”
  - $PPL_A = (\frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3}) - \frac{1}{5} = 3$
  - $PPL_B = (0.8 \times 0.8 \times 0.8 \times 0.8 \times 0.1) - \frac{1}{5} = 1.895$

- Thus, language model B is better than model A.

Implementation of N-Grams: Steps

1. Collect a corpus (text-based dataset).

2. Data preprocessing via regular expressions and tokenization to determine vocab & tokenized text.

3. Split corpus into training and test
   - Add special symbols and determine n-gram order (2,3,4,…).
   - Use **training data** to estimate conditional probabilities.
   - Reseve **test data** only for evaluation purposes.

4. Determine evaluation metric according to application.

5. Demonstrate/deploy model for the targeted domain.

Today

- N-Grams
- How to implement them
- **Practical considerations**

Acknowledgement: Images from https://web.stanford.edu/~jurafsky/slp3/3.pdf

Practical Considerations: Large N-Grams

Multiplying enough n-gram probabilities together would result in numerical underflow.

Adding in log space is equivalent to multiplying in linear space, so we combine log probabilities by adding them:

$$p1 * p2 * p3 * p4 = \exp(\log p1 + \log p2 + \log p3 + \log p4)$$

By adding log probabilities instead of multiplying them, we get results that are not as small.

We will use log to mean natural logarithm (ln) when the base is not specified; in Python math.log()

Log Probabilities: Example

Consider

$$
\begin{aligned}
&P(\texttt{<s> i want english food </s>}) \\
&\quad = P(\texttt{i}|\texttt{<s>})P(\texttt{want}|\texttt{i})P(\texttt{english}|\texttt{want}) \\
&\qquad\qquad P(\texttt{food}|\texttt{english})P(\texttt{</s>}|\texttt{food}) \\
&\quad = 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 \\
&\quad = 0.000031
\end{aligned}
$$

In log space we get

$$
\begin{aligned}
&P(< s > i\ want\ english\ food\ </s >) \\
&= -1.386 + (-1.109) + (-6.813) + (-0.693) + (-0.386) \\
&= -10.387
\end{aligned}
$$

Back to linear: $e^{-10.387} = math.\exp(-10.387) = 0.000031$

Practical Considerations: Smoothing

Problem with MLE: missing perfectly acceptable word sequences, leading to n-grams with zero probabilities.

I.e., particular n-grams that never appear in the training corpus, they occur in the test set.

Solution to avoid zero probabilities:

- **Laplace** smoothing: $$P_{\text{Laplace}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+1}{C(w_{n-1})+V}$$
- **Add-k** smoothing: $$P^*_{\text{Add-k}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+k}{C(w_{n-1})+kV}$$

The value $k \in \{0.5, 0.1, 0.01\}$ can be set manually and optimized using a devset (independent from test data)

Today

- **N-grams** are probabilistic language models.
  - Estimate conditional probabilities using MLE, with non-zero probabilities and in log-space.
  - Evaluate them using the metric ‘Perplexity’.

- **Applications**: Even though n-grams are not the state-of-the-art in language modelling, they are still very relevant these days due to their simplicity, scalability, and reliability (under certain conditions).

What's Next?

- Readings:
  - Jurafsky and Martin. Speech and Language Processing. Chapter 3, 2026. [recommended]
  - C. Buck, et al. "N -gram Counts and Language Models from the Common Crawl". LREC, 2014.

- Workshop **tomorrow**: Python code for training and evaluating n-gram language models.

- Next lecture: Text Classification.